문자열 다루기

PYTHON 자료형

◆ 문자열(String) 자료형

- 한 개 이상의 문자로 구성된 자료형
- ' 또는 "로 문자열 감싸기

```
print("Hello")

print('Python is Fun.')

print("₩'Life is too short~₩'")

print('₩"Life is too short~₩"')

print("Python's favorite food is perl")
```

```
Hello
Python is Fun.
'Life is too short~'
"Life is too short~"
Python's favorite food is perl
```

PYTHON 자료형

◆ 문자열(String) 자료형

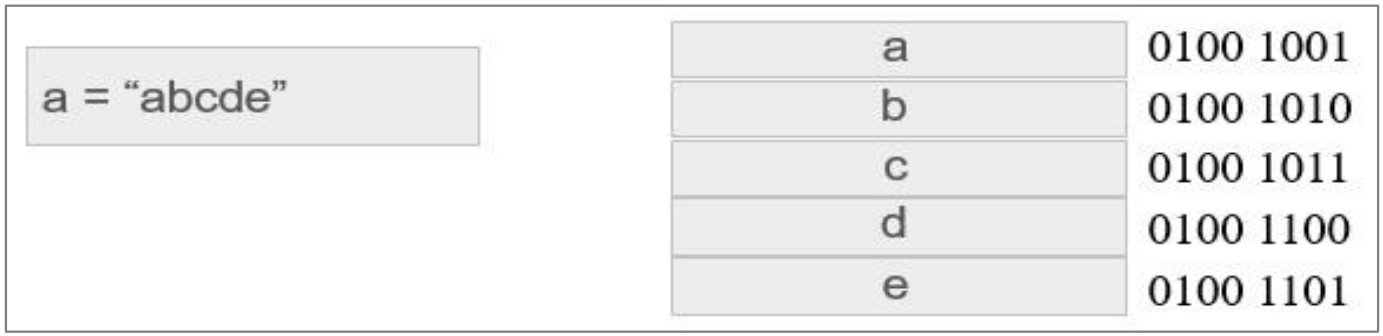
➢ 여러 줄 문자열

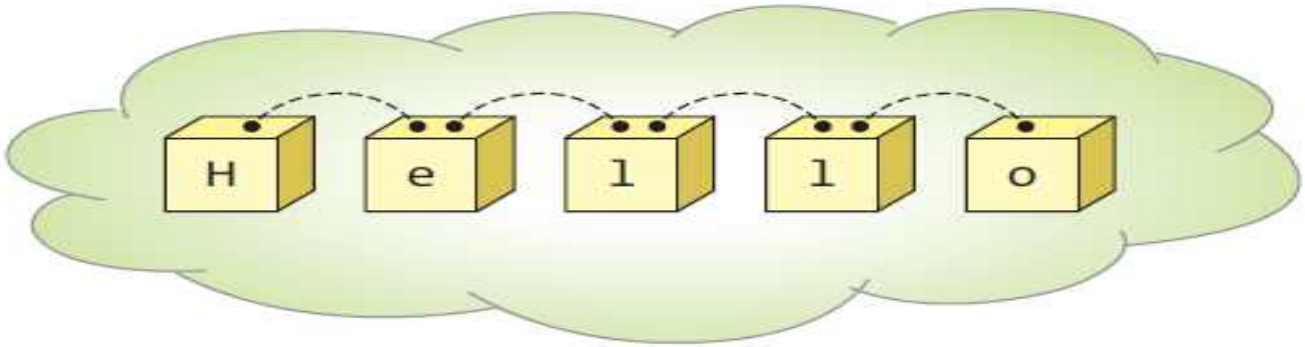
```
multiline="Life is too short₩nYou need python₩nPython is powerful language"
print(multiline)

multiline="""
 Life is too short
 You need python
 Python is powerful language
"""
print(multiline)
```

PYTHON 자료형

◆ 문자열(String) 자료형

➢ 메모리 공간 할당





◆ 문자열(String) 자료형

➢ 인덱싱(Indexing)

- 각 문자는 개별 주소를 가지며 주소로 값 가져오기

msg = 'Hello'

원소, 요소

H	e	l	l	o
0	1	2	3	4
-5	-4	-3	-2	-1

왼쪽부터 0~
오른쪽부터 -1~

문자열[번호]

◆ 문자열(String) 자료형

➢ 인덱싱(Indexing)

- 문자열, 리스트, 튜플 등등 자료형 타입에서 데이터 요소 하나하나에 번호를 부여하는 것
- 데이터 덩어리를 **하나씩 또는 일부를 사용하기 위한 방법**

0	1	2	3	4	5	6	7	8	9	10	11	12	13
H	**A**	**P**	**P**	**Y**		**N**	**E**	**W**		**Y**	**E**	**A**	**R**
-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

◆ 문자열(String) 자료형

➢ 인덱싱(Indexing)

```
Life is too short, You need Python
0          1          2          3
0123456789012345678901234567890123
```

```
>>> a[-0]
'L'
>>> a[-2]
'o'
>>> a[-5]
'y'
```

◆ 문자열(String) 자료형

➢ 슬라이싱(Slicing)

- 각 문자의 **인덱스를 기반으로** 문자 **잘라내기**

시작문자<= 문자열<끝문자

- 형식 ➔ **문자열[시작 번호 : 끝 번호]**

  - **주의! 끝 번호 포함 안됨**

◆ 문자열(String) 자료형

➢ 슬라이싱(Slicing)

- 형식 >> [시작번호 : 끝 번호]
- 주의 >> 끝 번호는 포함 안됨

[0 : **5**] 0 <= ~ <5

0	1	2	3	4	5	6	7	8	9	10	11	12	13
H	**A**	**P**	**P**	**Y**		**N**	**E**	**W**		Y	E	A	R
-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

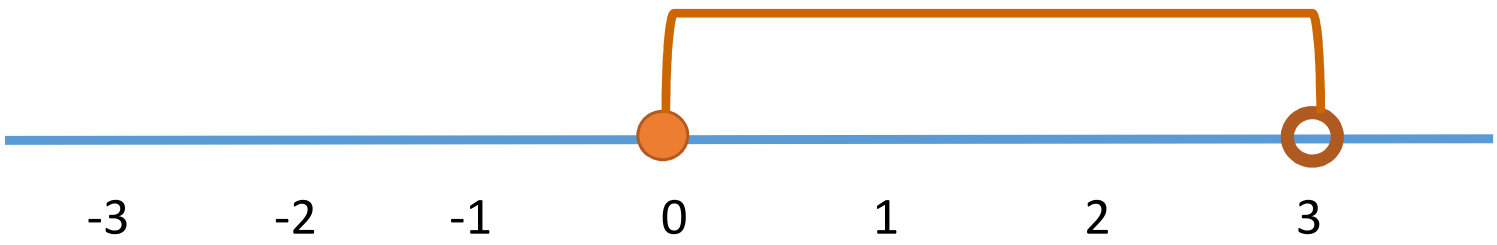
[-14 : **-9**]

◆ 문자열(String) 자료형

➢ 슬라이싱(Slicing)

- 형식 >> [시작번호이상 : 끝 번호 미만]
- 주의 >> 끝 번호는 포함 안됨

0, 1 , 2 => 0<= 숫자 <3
시작인덱스 이상<= <끝 인덱스



PYTHON 자료형

◆ 문자열(String) 자료형

➢ 슬라이싱(Slicing)

```
>>> a = "Life is too short, You need Python"
>>> a[0:4]
'Life'
>>> a[0:3]
'Lif'
>>> a[19:]
'You need Python'
>>> a[:17]
'Life is too short'
>>> a[:]
'Life is too short, You need Python'
>>> a[19:-7]
'You need'
```

PYTHON 자료형

◆ 문자열(String) 자료형

[문자 하나 하나씩 출력]

```
>>> Msg="Good-Luck"
>>> print( Msg[0], Msg[1], Msg[2])
>>> Msg[3]
>>> print(Msg)
```

PYTHON 자료형

◆ 문자열(String) 자료형

[일부 문자 출력]

```
>>> Msg="Good-Luck"
>>> len(Msg)          #문자열 길이
>>> Msg[0:3]
>>> Msg[3:]
>>> Msg[:-3]
>>> Msg[:]            #문자열 처음부터 끝까지
```

PYTHON 자료형

◆ 문자열(String) 자료형

[문자열 나누기]

```
>>> data="Happy New Year 2020"
>>> message=data[:14]
>>> message
>>> year=data[ len(message)+1 : ]
>>> year
```

PYTHON 자료형

◆ 문자열(String) 자료형

[특정 문자 바꾸기]

```
>>> data='Pithon'
>>> data[1] = 'y'     # Error
>>> data=data[:1]+'y'+data[2:]
>>> data
```

PYTHON 자료형

◆ 문자열(String) 자료형

➢ 문자열 ➔ 숫자 변환(캐스팅 Casting)

```
t = input("정수를 입력하세요: ")
x = int( t )

t = input("정수를 입력하세요: ")
y = int( t )
print(x+y)
```

<class **'str'**> **type**

<class 'int'> type

PYTHON 자료형

◆ 문자열(String) 자료형

➢ 숫자 ➔ 문자열 변환

```
>>> print(“나는 현재 “ + str(21) + “살이다.”)

Traceback (most recent call last):
File "<pyshell#1>", line 1, in <module>
print('나는 현재 ' + 21 + '살이다.')
TypeError: Can't convert 'int' object to str implicitly
```

◆ 문자열(String) 자료형

➢ 숫자 ➔ 문자열 변환

```
>>> print("나는 현재 "+ str(21) + "살이다.")
나는 현재 21살이다.

>>> print("원주율은 " + str(3.14) + "입니다.")
원주율은 3.14입니다.
```

◆ 문자열(String) 자료형

➢ 이스케이프 **문자**

- **특수 기능 수행** 문자
- 형식 ₩문자

이스케이프 문자	설 명
\n, ₩n	개행(Newline, 줄바꿈)
\t, ₩t	탭(Tab)
\0, ₩0	NULL 문자
\\, ₩₩	문자 '\'
\' , ₩'	단일 인용부호(')
\" , ₩"	이중 인용부호(")

PYTHON 자료형

◆ 문자열(String) 자료형

➢ 이스케이프 **문자**

```
print("오늘은 ₩'수요일₩' 입니다.")
print('오늘은 ₩'수요일₩' 입니다.')

print("오늘은 ₩n날씨가 춥군요.₩n")
print('₩n오늘은 ₩n날씨가 춥군요.₩n')

print("오늘은₩t날씨가₩t춥군요")

print("오늘은 ₩"크리스마스₩" 입니다.")
```

PYTHON 자료형

◆ 문자열(String) 자료형

➢ Raw String

- 문자열 내 **이스케이프문자 무시**
- 형식 ==> **r**"문자열"
- 경로, URL 문자열 처리에 자주 사용됨

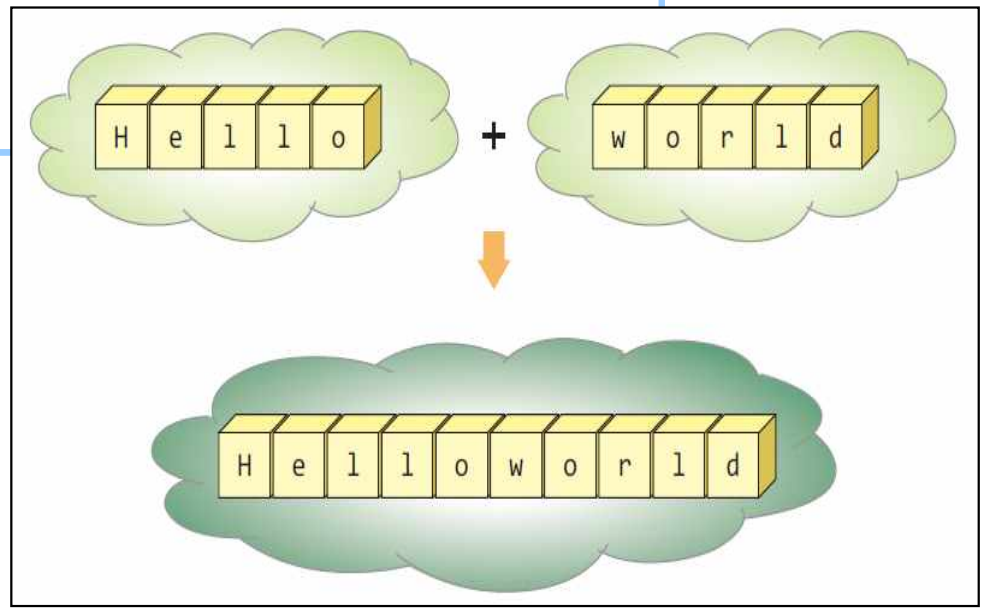
```
print( "오늘은 날씨가 너무₩n 좋군요₩₩" )

print( r"오늘은 날씨가 너무₩n 좋군요₩₩" )
```

PYTHON 자료형

◆ 문자열(String) 자료형

➢ 합치기 (Concatenation) ➔ + 연산자

```
>>> head = "Hello"
>>> tail = "World"
>>> print(head + tail)
```



PYTHON 자료형

◆ 문자열(String) 자료형

➢ 합치기 (Concatenation) ➔ + 연산자

```
msg="Good Luck"
name="Hong"
total=msg+name
print("id(msg) =>",   id(msg))
print("id(name) =>",  id(name))
print("id(total) =>", id(total))

# int 타입  + str 타입
print(str(2021)+"Happy New Year")
print("Happy New Year"+str(2021))
```

PYTHON 자료형

◆ 문자열(String) 자료형

➢ 곱하기 ➔ * 연산자

- '*'의미는 숫자 곱하기의 의미와는 다르게 사용
- '*'의미는 문자열의 **반복** 의미

```
>>> print("="* 5)
>>> print("My Program ")
>>> print("="* 5)
```

PYTHON 자료형

◆ 문자열(String) 자료형

➢ 변수값 포함 ➔ % 연산자

```
>>> price=10000
>>> print("상품 가격은 %s원입니다." %price)
```

◆ 문자열(String) 자료형

➢ in, not in 연산자

```
# a in B : a가 B안에 존재하면 True
print( "P" in "Price is 1000")
print( "p" in "Price is 1000")
print( "is" in "Price is 1000")

# a not in B : a가 B안에 존재하지 않으면 True
print( "is" not in "Price is 1000")
```

◆ 문자열(String) 자료형

➢ 포맷팅

- 원하는 형식의 문자열 생성하는 format()메서드

index=value 형식 ➔ "문자열 { index }". format(value , ...)

```
year=2020
month=11
day=6

msg="오늘은 {0}년 {1}월 {2}일 입니다.".format(year, month, day)
print("msg =>", msg)

msg="오늘은 {}년 {}월 {}일 입니다.".format(year, month, day)
print("msg =>", msg)
```

PYTHON 자료형

◆ 문자열(String) 자료형

➢ 포맷팅

- 원하는 형식의 문자열 생성하는 format()메서드

name=value 형식 ➔
"문자열 { name }". format(name=value , ...)

```
msg="{y}년 {m}월 {d}일 입니다.".format( d=25, y=2020, m=12)

print("msg =>", msg)
```

PYTHON 자료형

◆ 문자열(String) 자료형

➢ 포맷팅

분류	형태	설명
정렬	:<n	총자릿수n,<왼쪽정렬
	:>n	총자릿수n,>오른쪽정렬
	:^n	총자릿수n,^가운데정렬
공백채우기	:문자^n	총자릿수n,공백대신채울문자
소수점	:n.xf	총자릿수n,소수점표현자릿수x
'{','}'표현	{{}}	중괄호({ })그대로표현

◆ 문자열(String) 자료형

➢ 포맷팅

```
print(" {0:<10} 님! 좋은 하루되세요." .format("Happy") )
print(" {0:>10} 님! 좋은 하루 되세요." .format("Happy") )
print(" {0:^10} 님! 좋은 하루 되세요." .format("Happy") )
print(" {0:=^10} 님! 좋은 하루 되세요." .format("Happy") )
print(" {:->10.5f} 님! 좋은 하루 되세요." .format(12.12345678) )
print("그림_{:0>3}.jpg, 그림_{:0>3}.jpg 저장!".format(1, 123) )
```

```
안녕하세요. Happy     님! 좋은 하루되세요.
안녕하세요.     Happy 님! 좋은 하루 되세요.
안녕하세요.  Happy    님! 좋은 하루 되세요.
안녕하세요. ==Happy=== 님! 좋은 하루 되세요.
안녕하세요. --12.12346 님! 좋은 하루 되세요.
```

◆ 문자열(String) 자료형

➢ 메서드

함수	기능
count(문자)	문자열내문자개수세기
find(문자)	문자가처음나오는위치알려주기-1:존재하지않은문자
index(문자)/index(문자열)	문자또는문자열의위치알려줌,없을경우error발생
join(문자열)	문자열합치기
lstrip()	왼쪽공백지우기
rstrip()	오른쪽공백지우기
strip()	양쪽공백지우기
replace(이전문자열,바꿀문자열)	문자열내특정문자열치환
split(구분자)	문자열을구분자기준으로문자열나누어리스트에담기

◆ 문자열(String) 자료형

➢ 메서드

```
# replace() => 원본 문자열을 새로운 문자열로 변환
# replace(old, new, [count])
msg = 'AB-C-D-EFGH'
print("msg =>", msg)
print("msg.replace('-', '') =>", msg.replace('-', ''))
print("msg.replace('-', '', 2) =>", msg.replace('-', '',2))
print("msg =>", msg)
```

◆ 문자열(String) 자료형

➢ 메서드

```
# strip()  => 양쪽 공백 모두 제거
# lstrip() => 왼쪽 공백만 제거
# rstrip() => 오른쪽 공백만 제거
msg = '     1234 56   '
print("msg =>", msg)
print("msg.strip()=>", msg.strip())
print("msg.ltrip()=>", msg.lstrip())
print("msg.rtrip()=>", msg.rstrip())
print("msg =>", msg)
```

PYTHON 자료형

◆ 문자열(String) 자료형

➢ 메서드

```
# split() => 구분자를 기준으로 문자열 나누기
msg = '1/2/3/4/5/6'
data=msg.split('/')     # 나누어진 문자열 리스트에 담아 반환
print("msg =>", msg)
print("data =>", data, type(data))

#join() => 구분자 통해 각각의 리스트 요소들 순서대로 연결
sep = "-"
print(sep.join(data))
```

◆ 문자열(String) 자료형

➢ 메서드

```
msg="15 kim Daegu 010-222-1111"
data=msg.split(' ')
print("data =>", data, type(data))


sep = ","
print(sep.join(data))
```

◆ 문자열(String) 자료형

➢ 메서드

```
# find() => 문자열 내에서 문자열 찾아서 인덱스 반환
#          없으면 -1
msg = 'It`s always such a pleasure'
print("₩nmsg =>", msg)
print("msg.find('way') =>", msg.find('way'))
print("msg.find('such') =>", msg.find('such'))


print("msg.find('such', 13) =>", msg.find('such', 13))
```

◆ 문자열(String) 자료형

➢ 메서드

```
# index() => 문자열 내에서 문자열 찾아서 인덱스 반환
# #          없으면 Error 발생
print("msg.index('way') =>", msg.index('way'))
print("msg.index('such', 13) =>", msg.index('such', 13))
```